# 4-Bit ALU (Addition / Subtraction)

Design Explanation and Simulation Proof

CMOS implementation in Cadence Virtuoso - VLSI Lab, Track A


| Team member | ID |
|---|---|
| Abdullah Al Jubayer | 22201245 |
| H. M. Tahsin Sheikh | 22201243 |
| Shahrier Shimul | 22201227 |


| Result at a glance | |
|---|---|
| 1-bit add/subtract slice | 16 of 16 input combinations pass |
| 4-bit ALU (ADD and SUB) | 13 of 13 test cases pass, including carry-out and borrow |
| Simulation | Cadence Spectre, transient analysis, load 10 fF on every output |

Every screenshot in this document comes from the team's own Cadence session. Each is placed next to the statement it proves.

# 1. What the ALU does

The ALU takes two 4-bit numbers **A** and **B** and a 1-bit operation select **OP**. It gives back a 4-bit result **R** and one flag **OVF**. It has no clock and no memory: the outputs settle a short time after the inputs change.

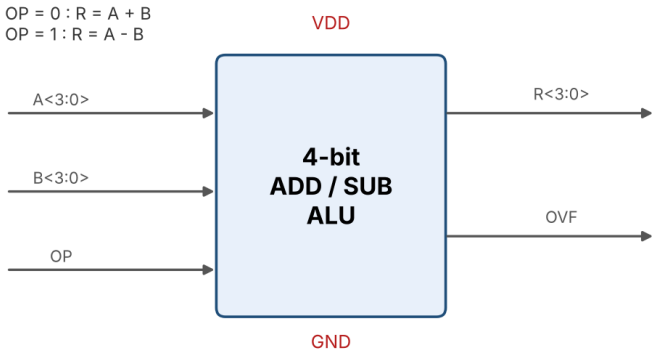



*Figure 1 - Top-level view. Pins: A0-A3, B0-B3, OP in; R0-R3, OVF out; vdd and gnd supply.*

| OP | Operation | How it is done | OVF = 1 means |
|---|---|---|---|
| 0 | R = A + B | B passes through unchanged, carry-in = 0 | Carry out: A + B is larger than 15 |
| 1 | R = A - B | B is inverted, carry-in = 1 (two's complement: A + B' + 1) | Borrow: A is smaller than B |

## The one idea behind the design

Subtraction reuses the adder. An XOR gate in front of each adder input flips B when OP = 1, and OP is also fed in as the first carry-in. That supplies the '+1' that turns the inverted B into a proper negative number. Four identical 1-bit slices are chained so each carry ripples into the next.

| Equation (for each bit i = 0 to 3) | Meaning |
|---|---|
| $B'_i = B_i$ xor OP | Conditionally invert B |
| $C_0$ = OP | Carry-in of slice 0 |
| $P_i = A_i$ xor $B'_i$ | Propagate |
| $R_i = P_i$ xor $C_i$ | Result bit |
| $C_{i+1}$ = [ $(A_i.B'_i)'$ . $(C_i.P_i)'$ ]' | Carry out (three NAND2 gates) |
| OVF = $C_4$ xor OP | Overflow in ADD, borrow in SUB |

## Why OVF = $C_4$ xor OP

In **ADD** (OP = 0) the flag is simply the carry out of the top bit, $C_4$. In **SUB** (OP = 1) the adder works on A + B' + 1, and a carry out of 1 means A was *not* smaller than B, so the result is valid. A carry out of 0 means a borrow happened. Flipping $C_4$ with OP makes OVF = 1 for 'something went wrong' in both modes.

| Example | B' + carry-in | C4 | R3..R0 | OVF |
|---|---|---|---|---|
| 5 + 3 (OP=0) | 0011, cin 0 | 0 | 1000 (8) | 0 |
| 15 + 1 (OP=0) | 0001, cin 0 | 1 | 0000 (0) | 1 (carry out) |
| 5 - 3 (OP=1) | 1100, cin 1 | 1 | 0010 (2) | 0 |
| 3 - 5 (OP=1) | 1010, cin 1 | 0 | 1110 (14 = -2) | 1 (borrow) |

# 2. Architecture

The design has three levels. Each level was built as a schematic, turned into a symbol, and used by the level above.

## 2.1 Full adder (gate level)

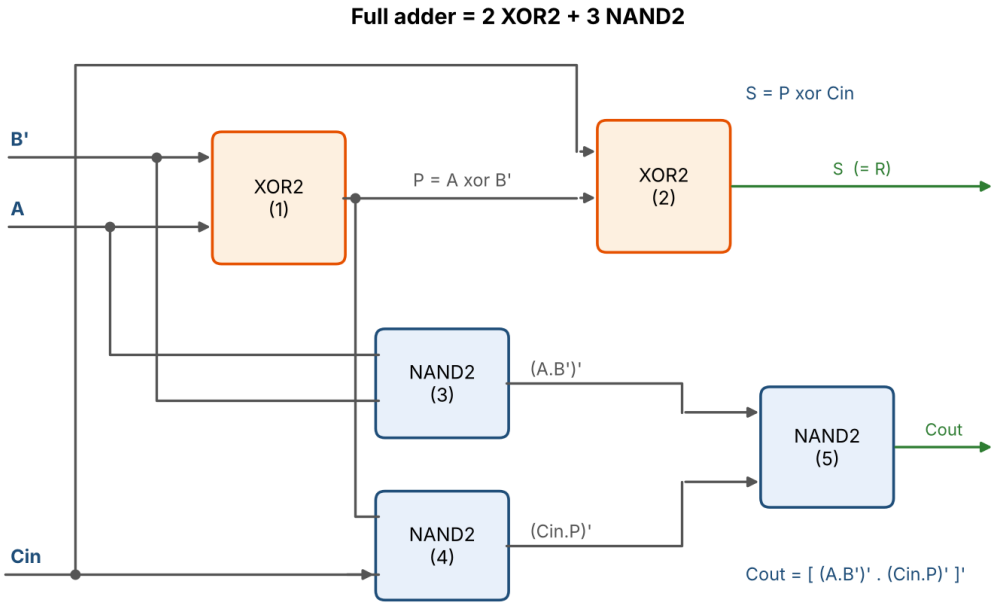



*Figure 2 - Full adder: two XOR2 gates make the sum, three NAND2 gates make the carry.*

## 2.2 One-bit add/subtract slice

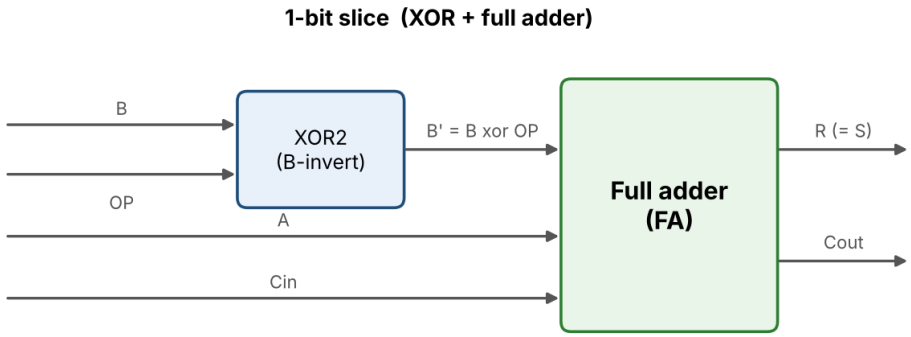



*Figure 3 - The slice: an XOR2 conditionally inverts B, then a full adder adds A, B' and the carry-in.*

## 2.3 Four slices and the OVF gate

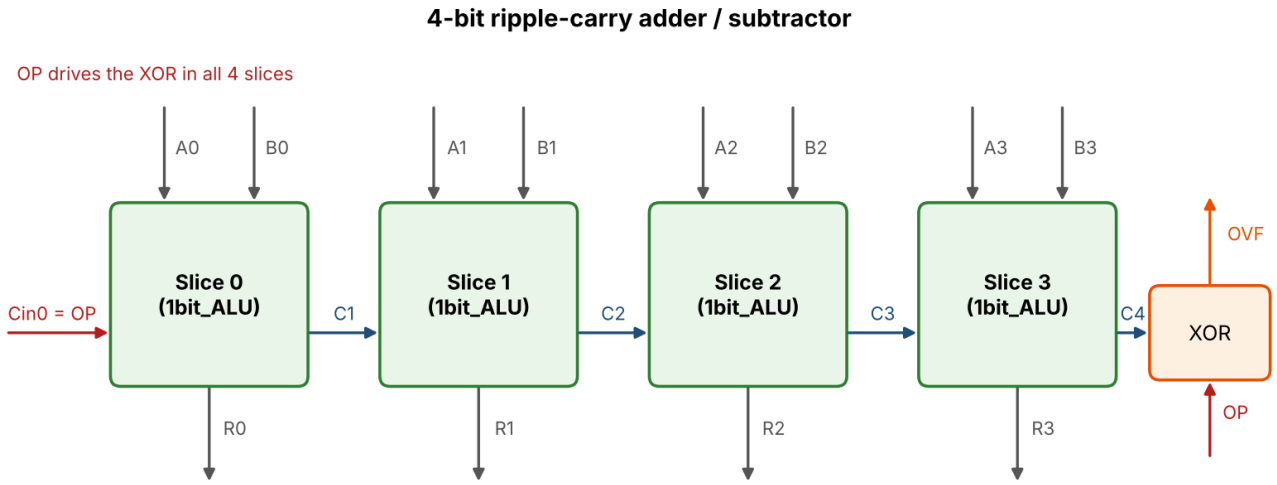



*Figure 4 - The ALU: four slices in a ripple-carry chain, with slice 0's carry-in tied to OP and one XOR2 for OVF.*

| Level | Contents | Gates | Transistors (static CMOS, as proposed) |
|---|---|---|---|
| Full adder | 2 XOR2 + 3 NAND2 | 5 | 36 |
| 1-bit slice | XOR2 + full adder | 6 | 48 |
| 4-bit ALU | 4 slices + OVF XOR2 | 25 | 204 |

The transistor counts come from the proposal's gate decomposition (XOR2 = 12, NAND2 = 4 transistors). They apply as long as the XOR2 and full-adder cells follow that structure.

### Critical path

The slowest path is the carry rippling from slice 0 to $C_4$. With A = 1111 and $B_0$ toggling in ADD mode, the change passes through 9 gate levels to $R_3$ and 11 to OVF. That is why the outputs are read at the end of each 10 ns window, after everything has settled.

# 3. Building blocks (own cells reused)

The XOR2 and the full adder were created earlier as separate cells and are used unchanged inside the slice.

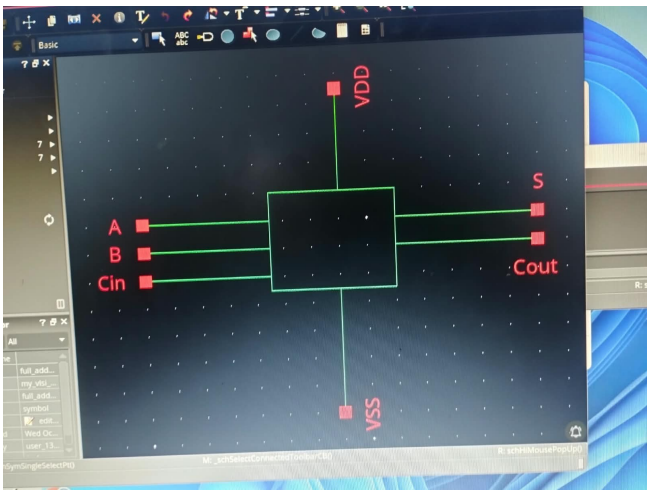



*Figure 5 - Full-adder symbol: inputs A, B, Cin; outputs S, Cout; supply VDD and VSS.*

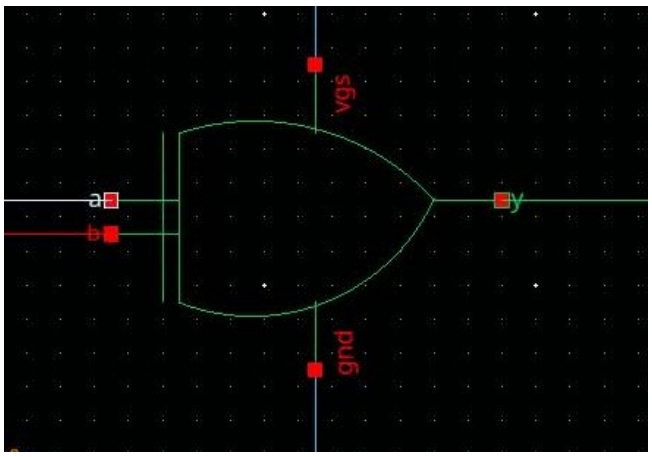



*Figure 6 - XOR2 symbol: inputs a, b; output y; supply vgs and gnd.*

# 4. One-bit slice

## 4.1 Schematic and symbol

Bin goes into the XOR2 together with OP. The XOR2 output y is the inverted-or-not B' and feeds the full adder's B input. A and Cin go straight to the full adder. The full adder's S output is the slice result R, and Cout leaves the slice.

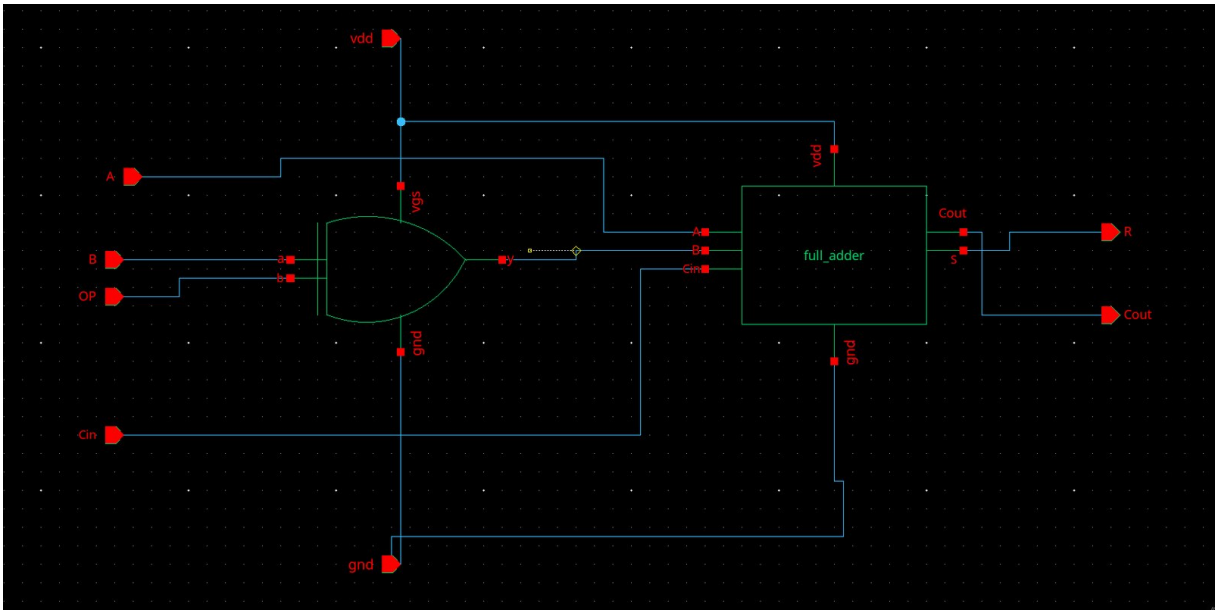



*Figure 7 - Slice schematic (library jubayer, cell 1bit_ALU). Pins: A, B, Cin, OP, R, Cout, vdd, gnd.*

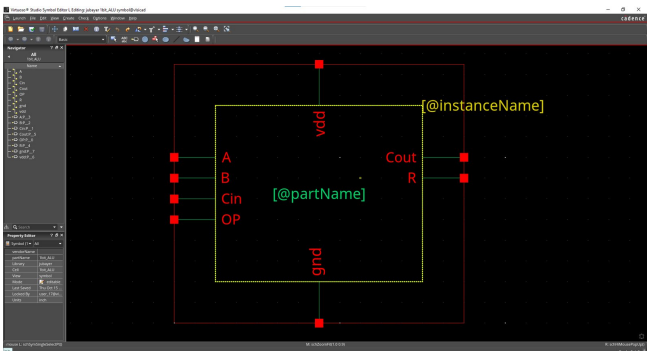



*Figure 8 - Symbol editor: A, B, Cin, OP on the left; Cout, R on the right; vdd on top, gnd below.*

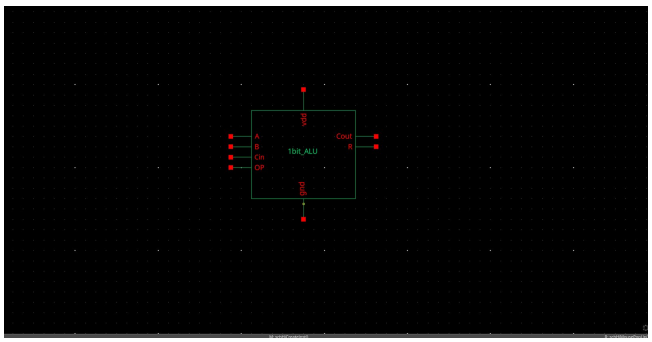


*Figure 9 - The 1bit_ALU symbol placed in a test schematic.*

## 4.2 Test setup and expected behaviour

The slice was tested on all 16 input combinations, 10 ns each, for 160 ns in one transient run. A toggles every 10 ns, B every 20 ns, Cin every 40 ns, and OP is high from 80 ns. Each output carries a 10 fF load to ground.

| Case | Window (ns) | OP | Cin | B | A | Expected R | Expected Cout | Simulated | Result |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 0-10 | 0 | 0 | 0 | 0 | 0 | 0 | R=0, Cout=0 | PASS |
| 2 | 10-20 | 0 | 0 | 0 | 1 | 1 | 0 | R=1, Cout=0 | PASS |
| 3 | 20-30 | 0 | 0 | 1 | 0 | 1 | 0 | R=1, Cout=0 | PASS |
| 4 | 30-40 | 0 | 0 | 1 | 1 | 0 | 1 | R=0, Cout=1 | PASS |
| 5 | 40-50 | 0 | 1 | 0 | 0 | 1 | 0 | R=1, Cout=0 | PASS |
| 6 | 50-60 | 0 | 1 | 0 | 1 | 0 | 1 | R=0, Cout=1 | PASS |
| 7 | 60-70 | 0 | 1 | 1 | 0 | 0 | 1 | R=0, Cout=1 | PASS |
| 8 | 70-80 | 0 | 1 | 1 | 1 | 1 | 1 | R=1, Cout=1 | PASS |
| 9 | 80-90 | 1 | 0 | 0 | 0 | 1 | 0 | R=1, Cout=0 | PASS |
| 10 | 90-100 | 1 | 0 | 0 | 1 | 0 | 1 | R=0, Cout=1 | PASS |
| 11 | 100-110 | 1 | 0 | 1 | 0 | 0 | 0 | R=0, Cout=0 | PASS |
| 12 | 110-120 | 1 | 0 | 1 | 1 | 1 | 0 | R=1, Cout=0 | PASS |
| 13 | 120-130 | 1 | 1 | 0 | 0 | 0 | 1 | R=0, Cout=1 | PASS |
| 14 | 130-140 | 1 | 1 | 0 | 1 | 1 | 1 | R=1, Cout=1 | PASS |
| 15 | 140-150 | 1 | 1 | 1 | 0 | 1 | 0 | R=1, Cout=0 | PASS |
| 16 | 150-160 | 1 | 1 | 1 | 1 | 0 | 1 | R=0, Cout=1 | PASS |

Expected values follow R = A xor (B xor OP) xor Cin and Cout = majority(A, B xor OP, Cin). The 'Simulated' column is what the waveforms in the next figures show at the end of each window.

## 4.3 Waveforms (proof)

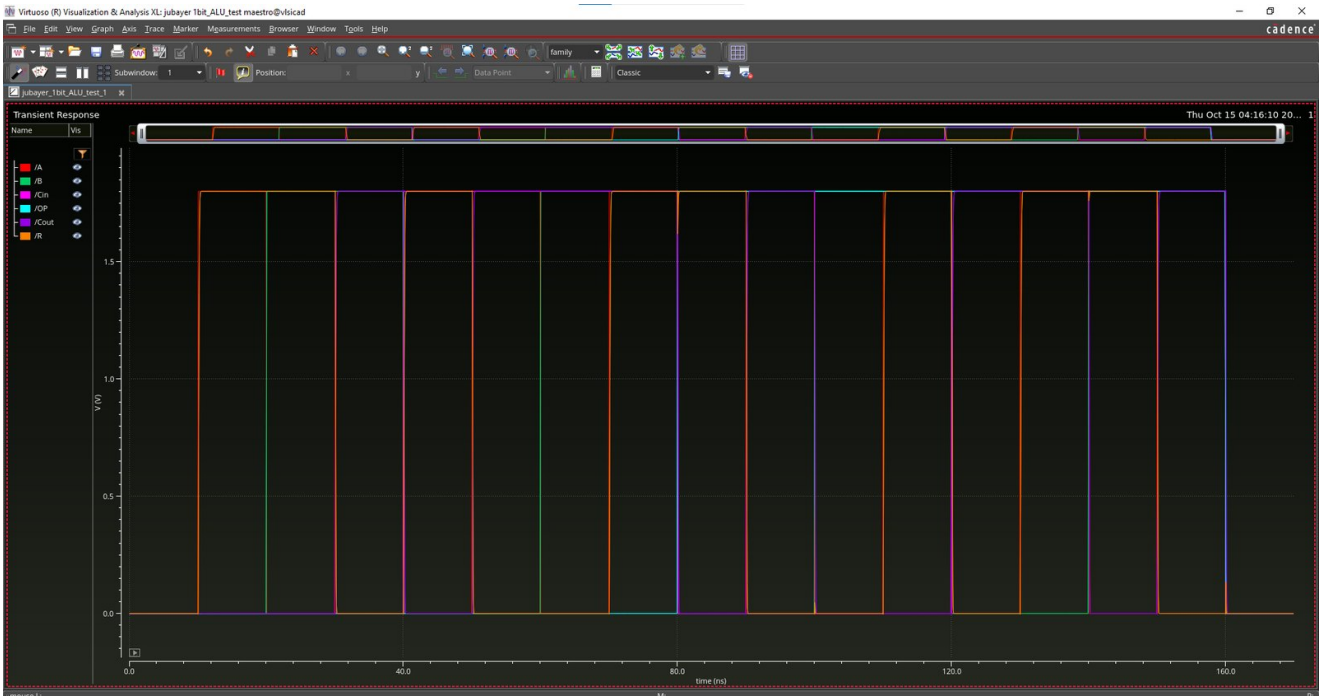



*Figure 10 - All six signals of the slice testbench over 160 ns.*

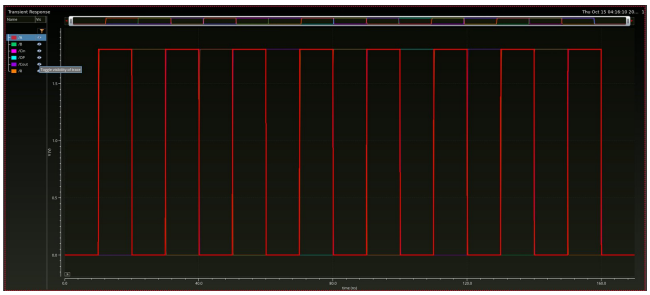


*Figure 11 - A: toggles every 10 ns.*

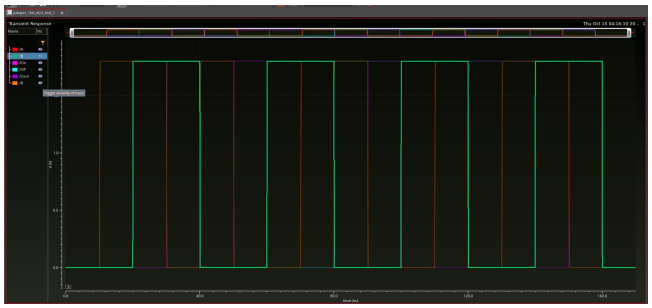


*Figure 12 - B: toggles every 20 ns.*

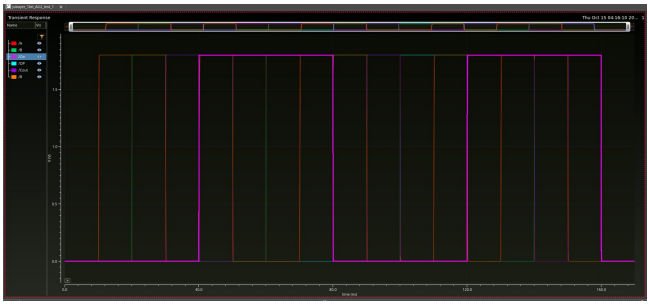


*Figure 13 - Cin: toggles every 40 ns.*

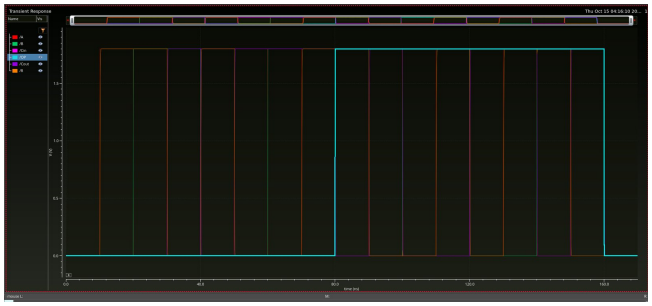


*Figure 14 - OP: low to 80 ns, high after.*

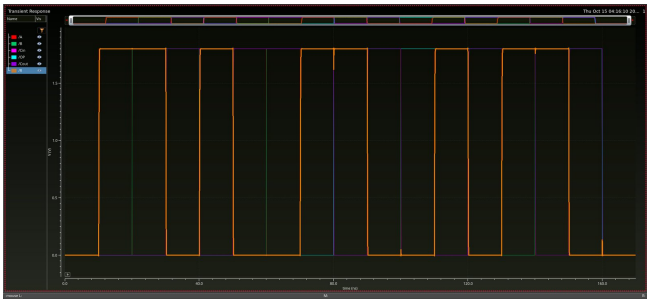


*Figure 15 - Output R (sum).*

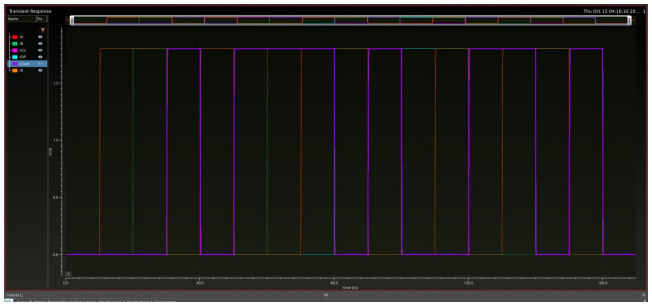


*Figure 16 - Output Cout (carry).*

Reading the waveforms: R equals A xor (B xor OP) xor Cin, so it flips whenever an odd number of those inputs is high, and Cout goes high when at least two of A, B', Cin are high. All 16 windows match the table

above.

# 5. Four-bit ALU

## 5.1 Symbol and testbench

Four copies of the slice plus the OVF XOR2 form the ALU. Its symbol exposes exactly the interface in the proposal.

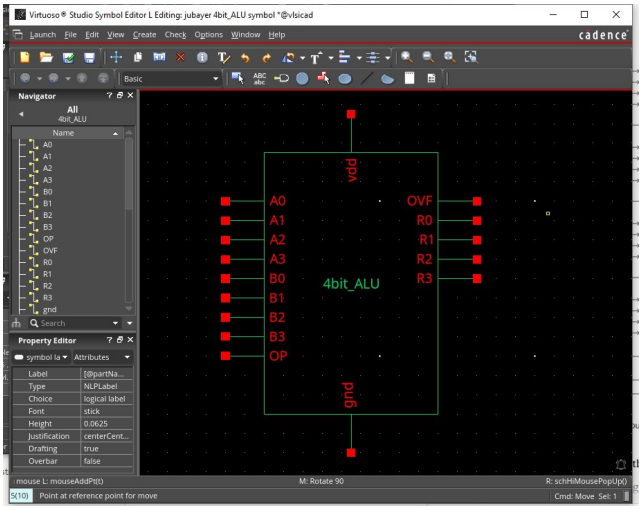



*Figure 17 - 4bit_ALU symbol: A0-A3, B0-B3, OP in; OVF, R0-R3 out; vdd and gnd.*

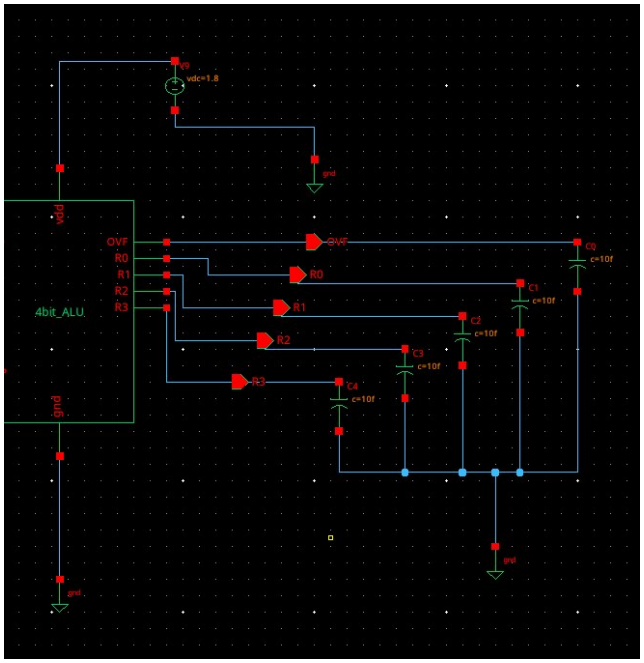



*Figure 18 - Output side of the testbench: 1.8 V supply (V9) and a 10 fF load on R0-R3 and OVF.*

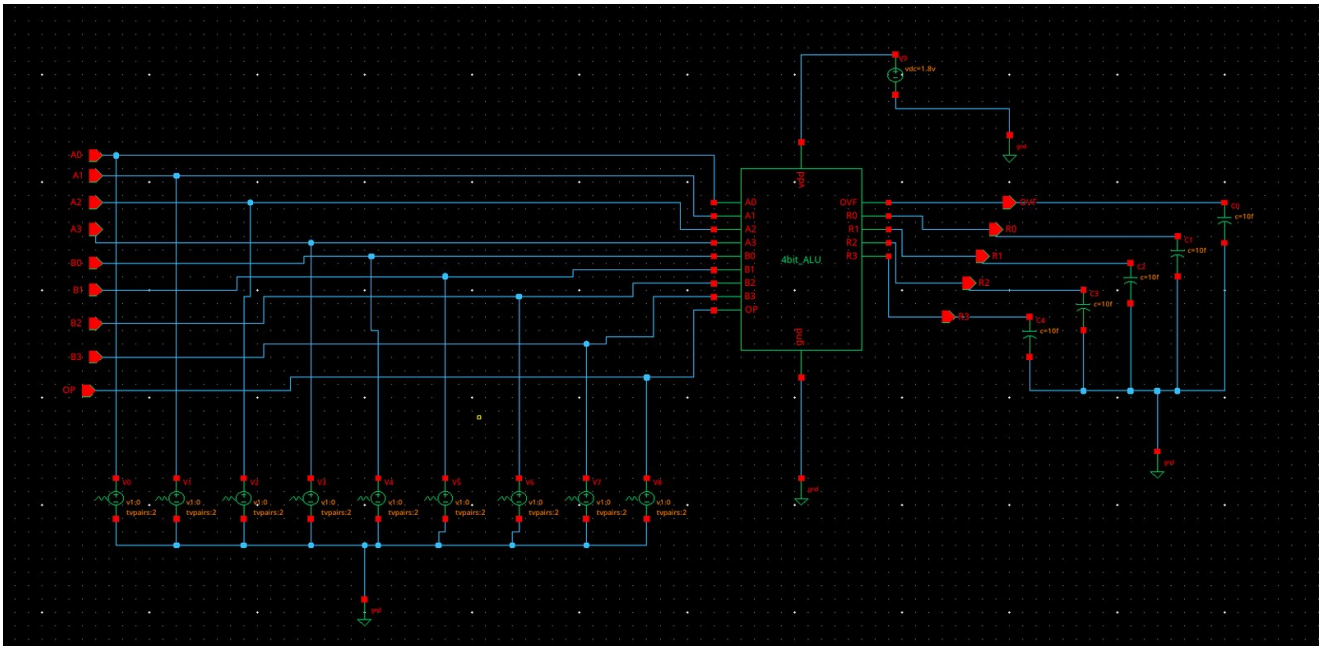


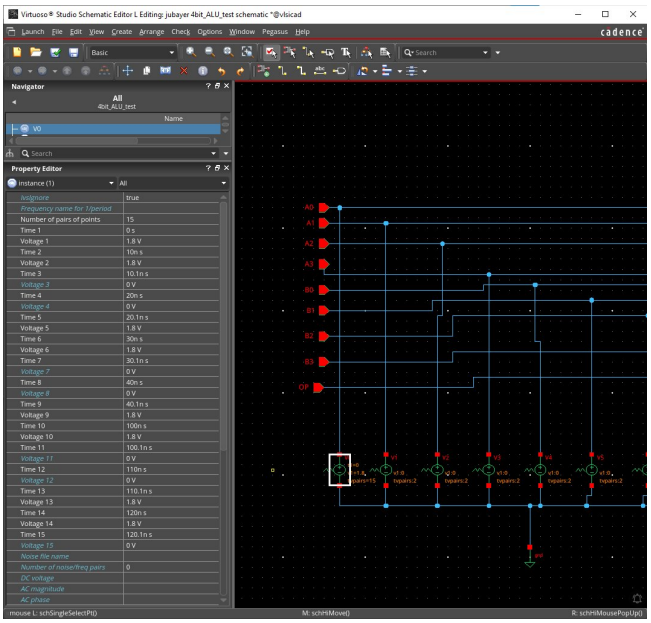

*Figure 19 - 4bit_ALU_test: nine vpwl sources (V0-V8) drive A0-A3, B0-B3 and OP; V9 is the supply; five 10 fF caps load the outputs.*

## 5.2 Stimulus: the vpwl sources

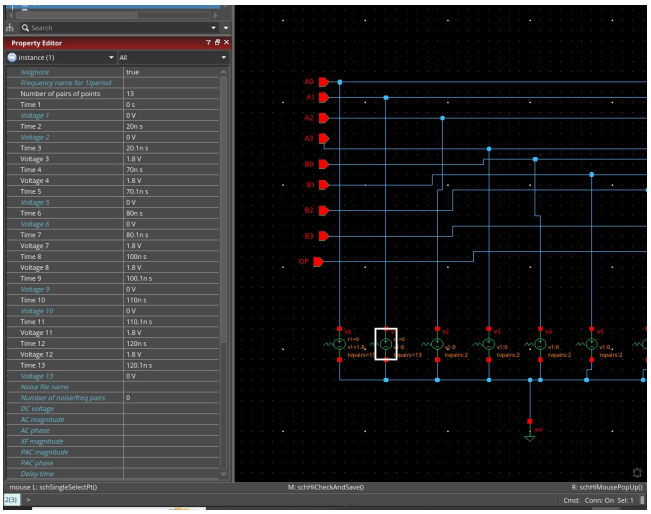
Each input is a piecewise-linear source. Every level change is entered as two points: the old level at the change time, then the new level 0.1 ns later, so the edges are clean. Case *k* lasts from (k-1) x 10 ns to k x 10 ns, and the transient runs to 150 ns.

| Source | Drives | Pairs | Waveform shape (ns) |
|---|---|---|---|
| V0 | A0 | 15 | high 0-10, low 10-20, high 20-30, low 30-40, high 40-100, low 100-110, high 110-120, low after 120 |
| V1 | A1 | 13 | low 0-20, high 20-70, low 70-80, high 80-100, low 100-110, high 110-120, low after 120 |
| V2 | A2 | 11 | high 0-10, low 10-20, high 20-30, low 30-40, high 40-100, low after 100 |
| V3 | A3 | 9 | low 0-30, high 30-60, low 60-90, high 90-110, low after 110 |
| V4 | B0 | 13 | high 0-10, low 10-20, high 20-60, low 60-70, high 70-90, low 90-100, high after 100 |
| V5 | B1 | 7 | high 0-10, low 10-50, high 50-90, low after 90 |
| V6 | B2 | 17 | low 0-30, high 30-40, low 40-50, high 50-60, low 60-80, high 80-90, low 90-110, high 110-120, low after 120 |
| V7 | B3 | 5 | low 0-50, high 50-60, low after 60 |
| V8 | OP | 3 | low 0-60, high after 60 |

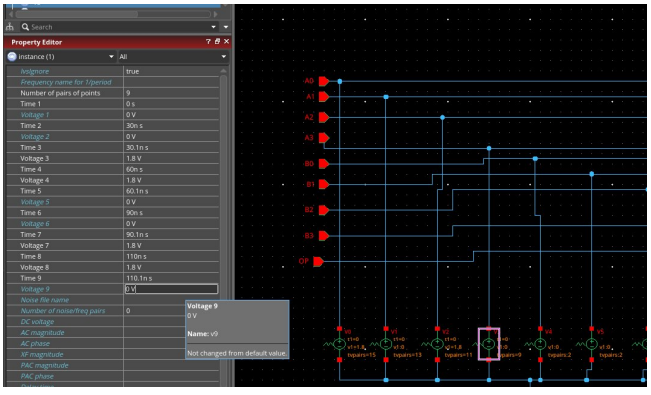
Property-editor screenshots of each source (proof that these values are what the simulator received). V2 is shown with its fix in Section 7.



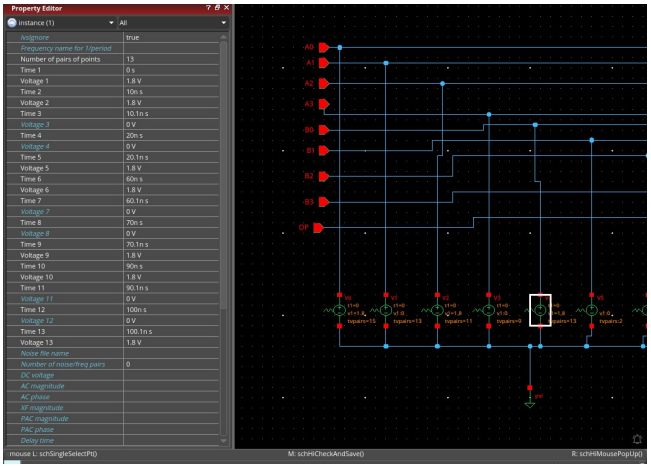
*V0 = A0, 15 pairs.*



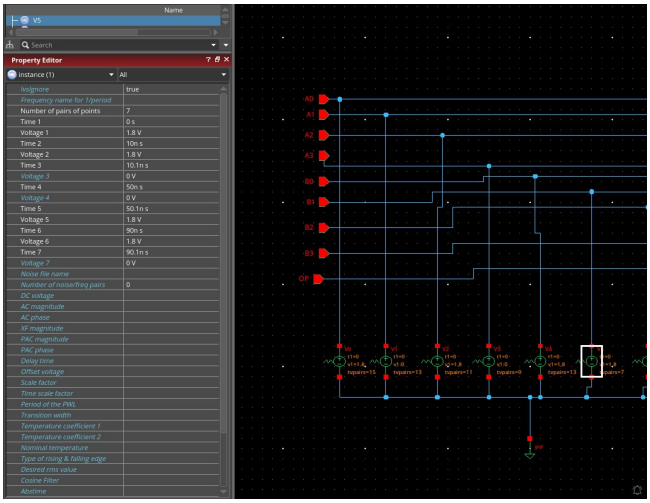
*V1 = A1, 13 pairs.*



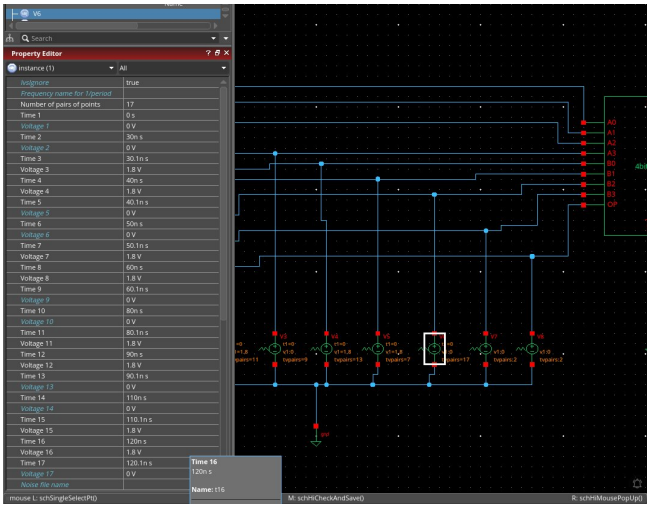
*V3 = A3, 9 pairs.*



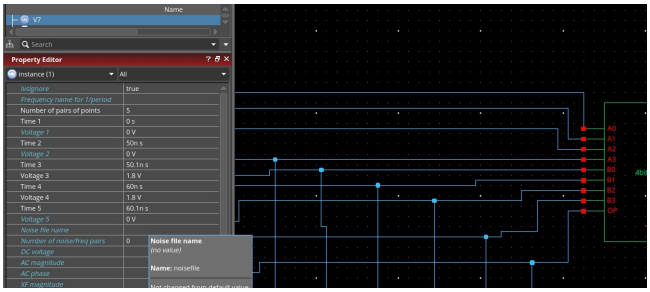
*V4 = B0, 13 pairs.*



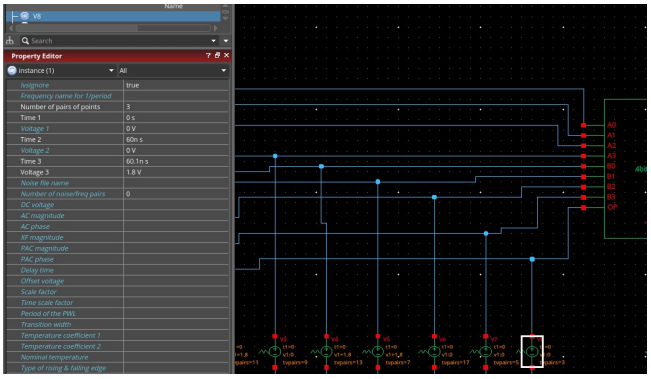
*V5 = B1, 7 pairs, voltages 0 / 1.8 V (after the fix).*



*V6 = B2, 17 pairs.*



*V7 = B3, 5 pairs.*



*V8 = OP, 3 pairs: low to 60 ns, high after.*

## 5.3 Input waveforms

The full-run plot below shows every signal over 0-150 ns. The per-signal views that follow highlight one trace at a time, so each input can be checked against its vpwl table.

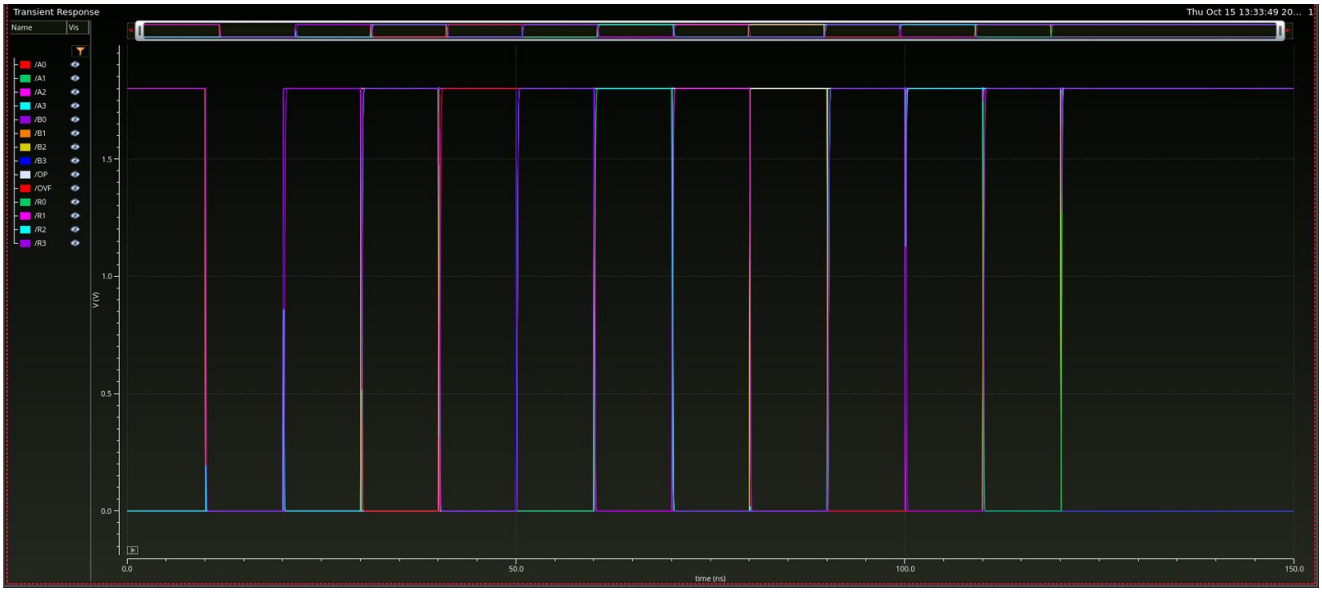



*Figure 20 - All inputs and outputs, 0 to 150 ns.*

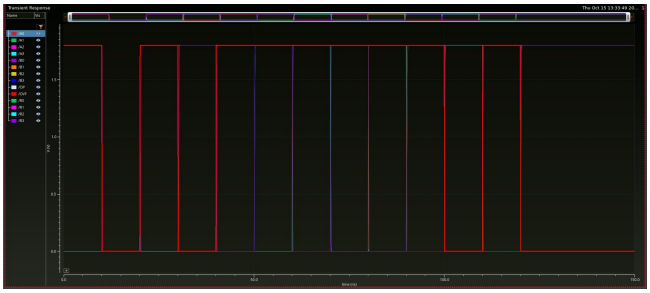


*Figure 21 - A0 (highlighted)*

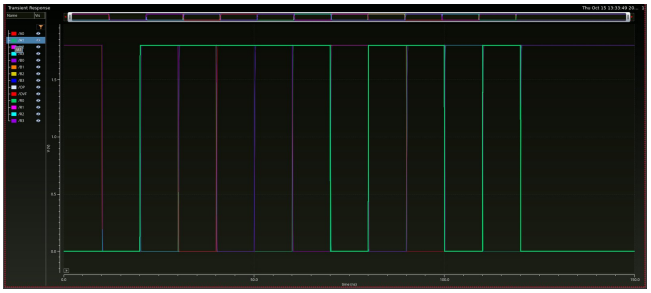


*Figure 22 - A1*

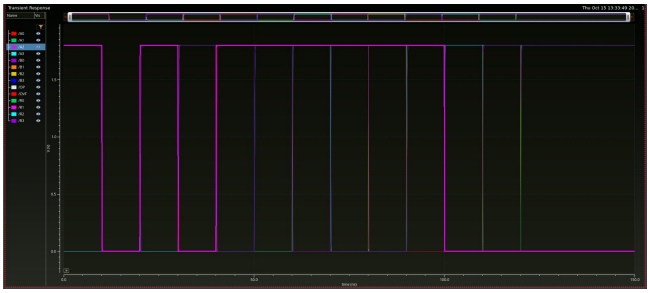


*Figure 23 - A2*

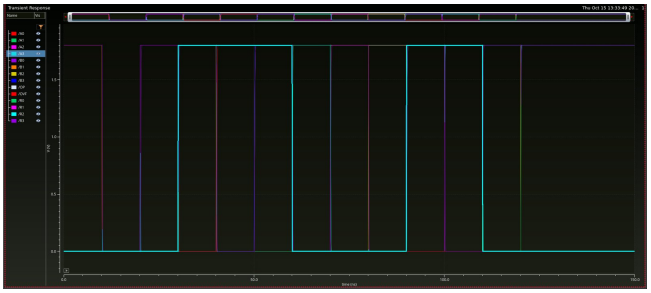


*Figure 24 - A3*

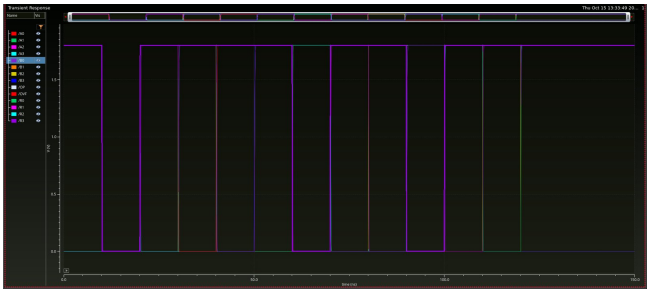


*Figure 25 - B0*

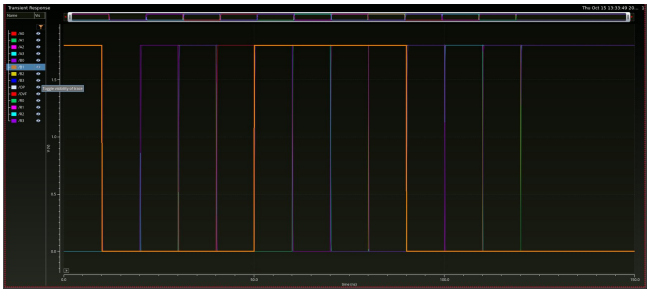


*Figure 26 - B1*

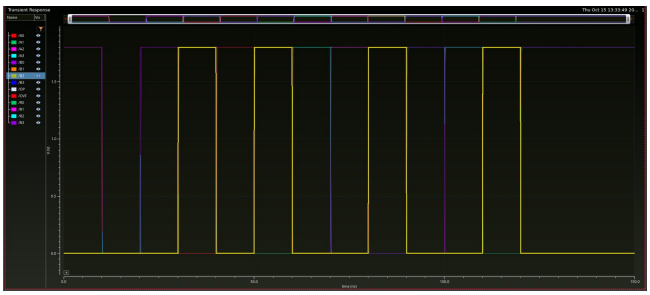


*Figure 27 - B2*

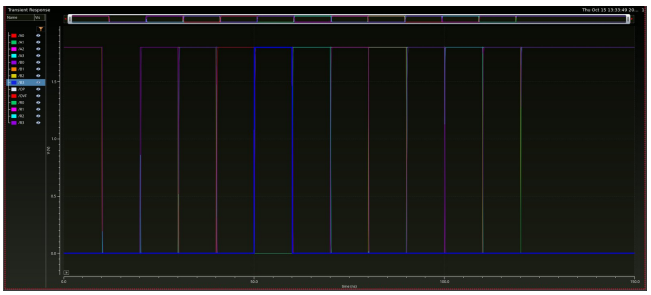


*Figure 28 - B3*

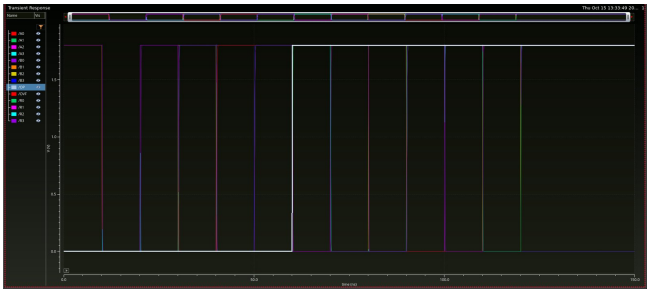


*Figure 29 - OP: low to 60 ns (ADD), high after (SUB)*

## 5.4 Output waveforms

Each output is shown on its own. Read each one near the end of its 10 ns window; short glitches at the start of a window are normal static-CMOS behaviour when several inputs change at once.

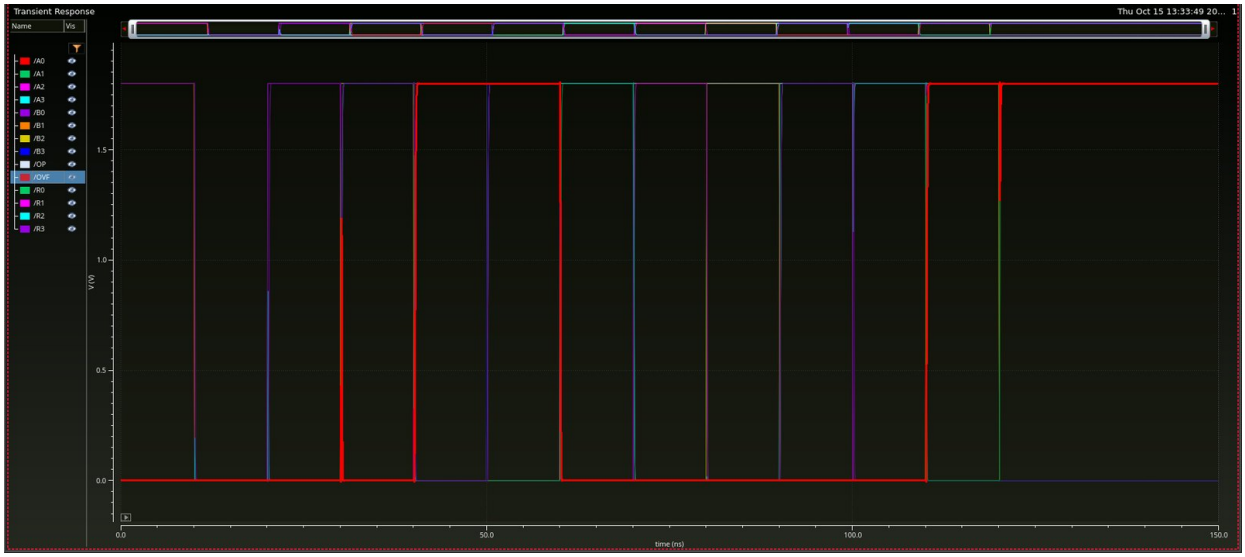



*Figure 30 - OVF: high in cases 5-6 (carry out in ADD) and cases 12-13 (borrow in SUB, from 110 ns on).*

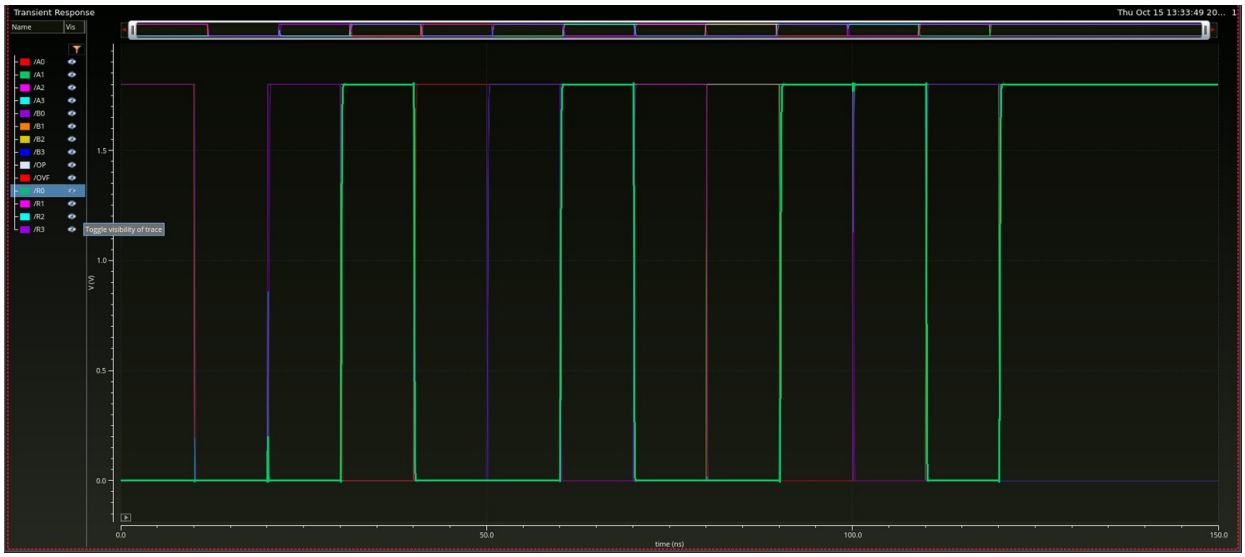



*Figure 31 - R0.*

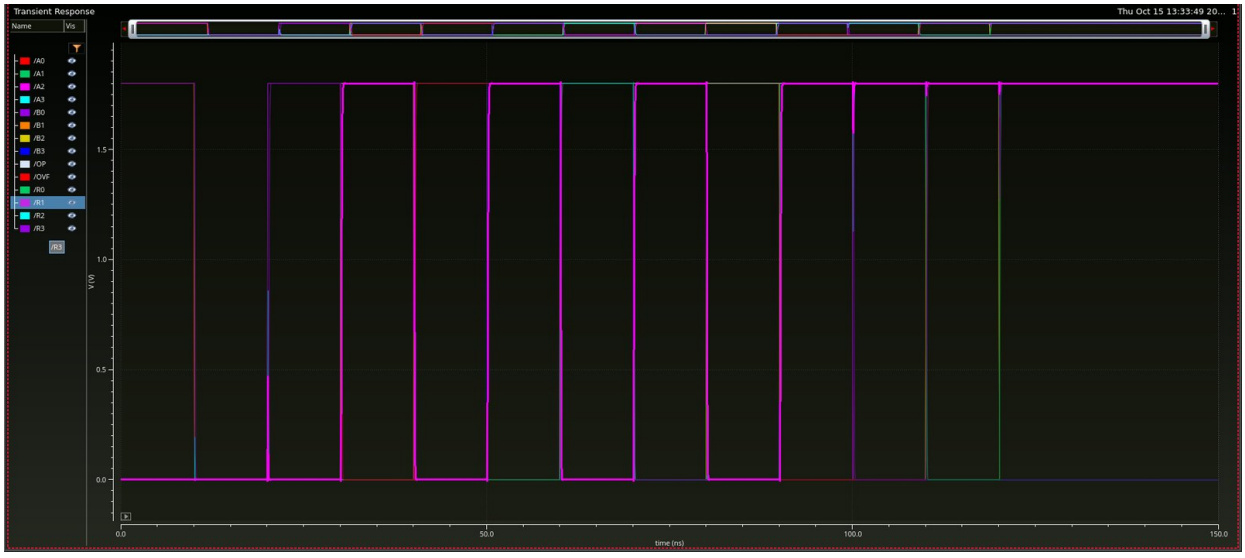



*Figure 32 - R1.*

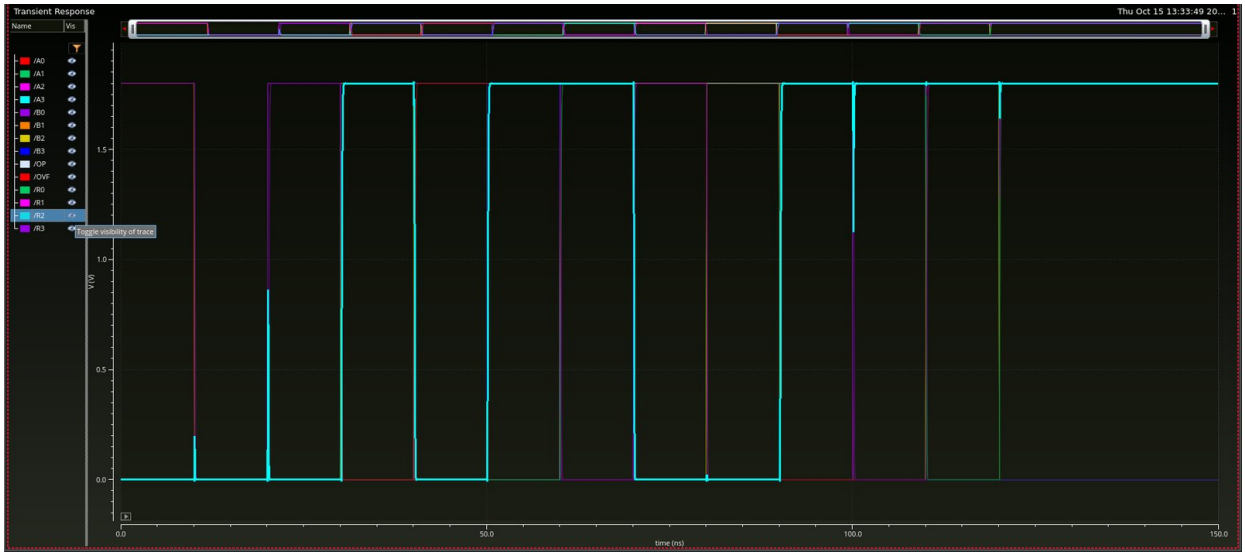



*Figure 33 - R2.*

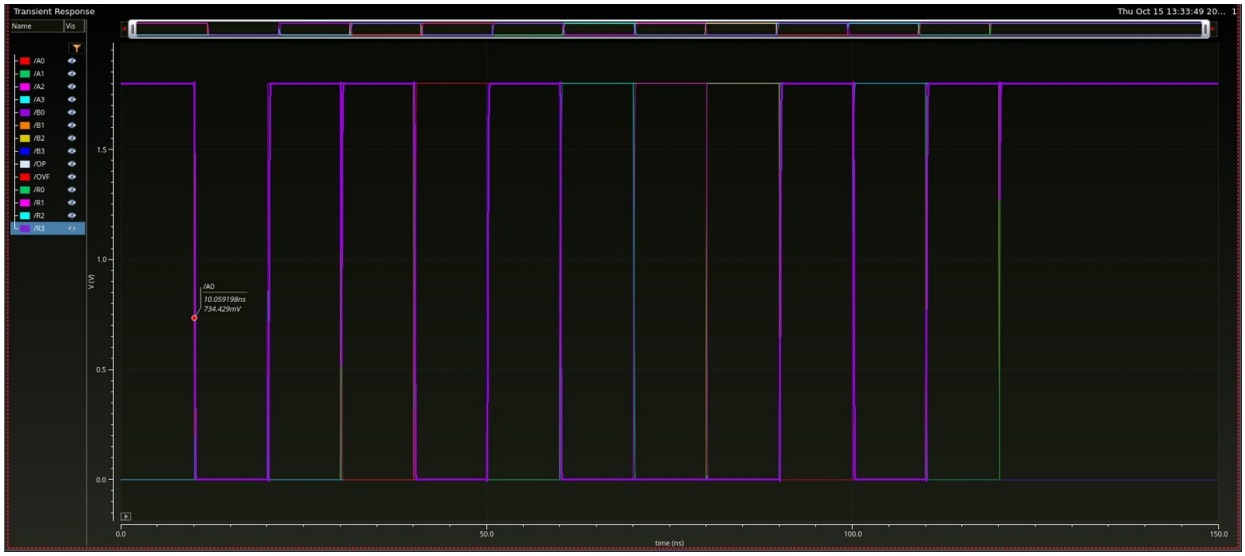



*Figure 34 - R3.*

## 5.5 Expected versus simulated results (13 cases)

Expected values were computed before simulation from R = A +/- B and the OVF rule. The simulated values are read from the output waveforms above at the end of each window. All 13 agree.

| Case | Window (ns) | OP | A | B | Operation | Expected R3..R0 | Expected OVF | Simulated R3..R0 | Simulated OVF | Result |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0-10 | 0 | 0101 | 0011 | 5 + 3 = 8 | 1000 | 0 | 1000 | 0 | PASS |
| 2 | 10-20 | 0 | 0000 | 0000 | 0 + 0 = 0 | 0000 | 0 | 0000 | 0 | PASS |
| 3 | 20-30 | 0 | 0111 | 0001 | 7 + 1 = 8 | 1000 | 0 | 1000 | 0 | PASS |
| 4 | 30-40 | 0 | 1010 | 0101 | 10 + 5 = 15 | 1111 | 0 | 1111 | 0 | PASS |
| 5 | 40-50 | 0 | 1111 | 0001 | 15 + 1 -> 0 (carry) | 0000 | 1 | 0000 | 1 | PASS |
| 6 | 50-60 | 0 | 1111 | 1111 | 15 + 15 -> 14 (carry) | 1110 | 1 | 1110 | 1 | PASS |
| 7 | 60-70 | 1 | 0111 | 0010 | 7 - 2 = 5 | 0101 | 0 | 0101 | 0 | PASS |
| 8 | 70-80 | 1 | 0101 | 0011 | 5 - 3 = 2 | 0010 | 0 | 0010 | 0 | PASS |
| 9 | 80-90 | 1 | 0111 | 0111 | 7 - 7 = 0 | 0000 | 0 | 0000 | 0 | PASS |
| 10 | 90-100 | 1 | 1111 | 0000 | 15 - 0 = 15 | 1111 | 0 | 1111 | 0 | PASS |
| 11 | 100-110 | 1 | 1000 | 0001 | 8 - 1 = 7 | 0111 | 0 | 0111 | 0 | PASS |
| 12 | 110-120 | 1 | 0011 | 0101 | 3 - 5 -> 14 (borrow) | 1110 | 1 | 1110 | 1 | PASS |
| 13 | 120-130 | 1 | 0000 | 0001 | 0 - 1 -> 15 (borrow) | 1111 | 1 | 1111 | 1 | PASS |

Shaded rows are the boundary cases: carry out in ADD (cases 5 and 6) and borrow in SUB (cases 12 and 13). A sanity check that holds in every case, for both operations: R0 = A0 xor B0.

### What the cases cover

- Zero operands (case 2) and all-ones operands (cases 5, 6, 10).
- Result wrap-around in ADD (15 + 1 = 0 with OVF = 1; 15 + 15 = 14 with OVF = 1).
- Equal operands in SUB giving zero (case 9: 7 - 7 = 0, no borrow).
- Borrow in SUB (case 12: 3 - 5 = 1110 = -2; case 13: 0 - 1 = 1111 = -1).

# 6. Problems found and fixed during the build

These screenshots are kept because they show the checks that were done. Each problem was found, understood and corrected before the final results above were produced.

## 6.1 Slice wiring

The first slice put the XOR on OP and A, and sent B straight to the adder. That inverts the wrong operand, so subtraction would compute the wrong answer. The corrected version (Figure 7) puts the XOR on B and OP and feeds its output into the adder's B input.

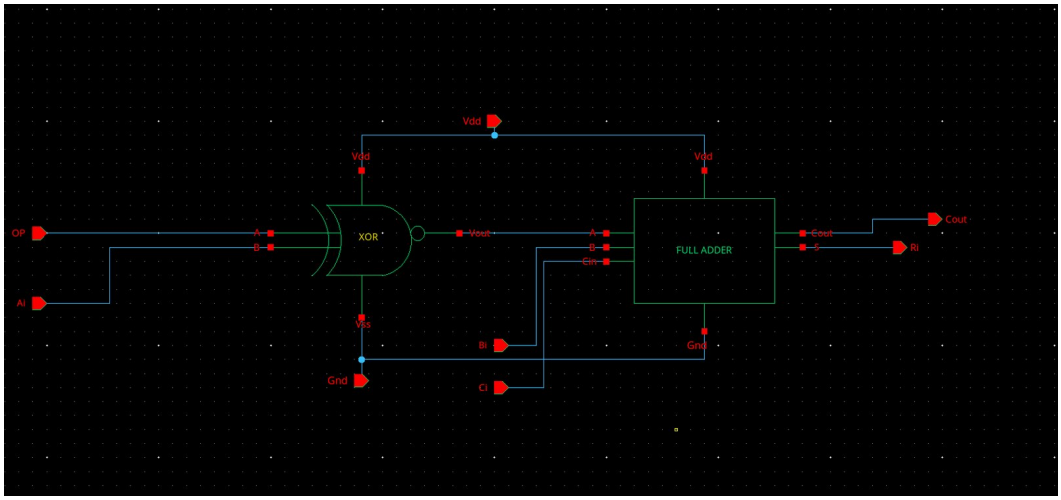



*Figure 35 - First attempt (wrong): XOR on OP and A, B bypassing the XOR.*

## 6.2 Load capacitors

Two early wirings were wrong: a capacitor placed in series with an output, and wires attached to a pin outline instead of its connection point. Each capacitor must hang from the output net to ground, with a junction dot where the wire joins.

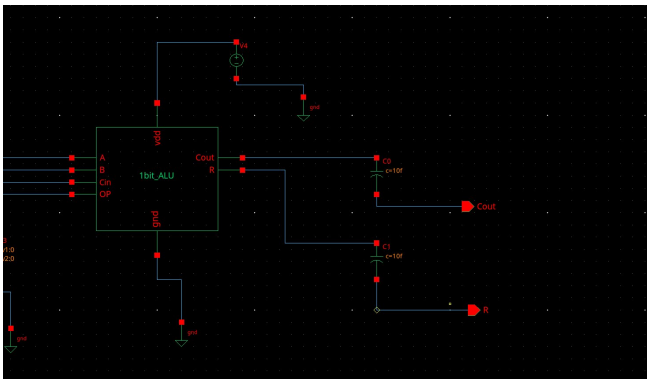



*Figure 36 - Capacitors in series with the outputs (wrong).*

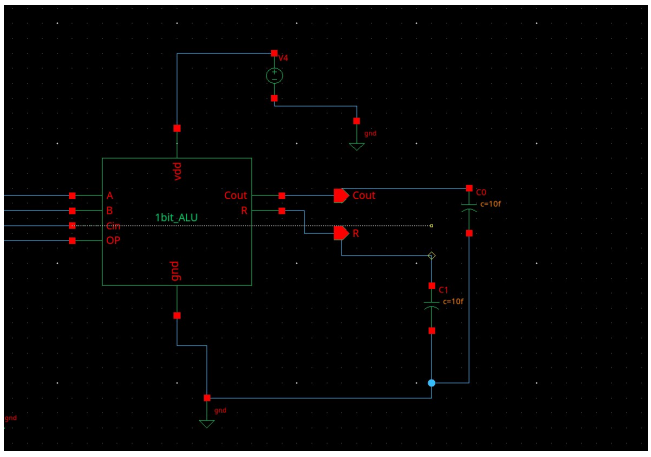
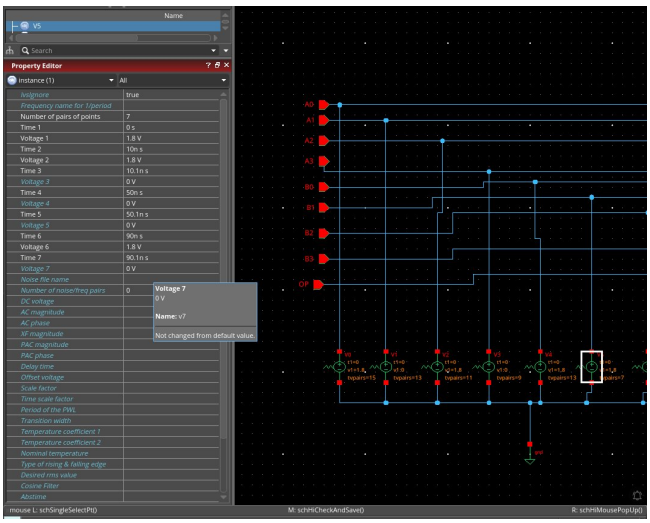



*Figure 37 - Capacitor wire not joined to its net (wrong).*

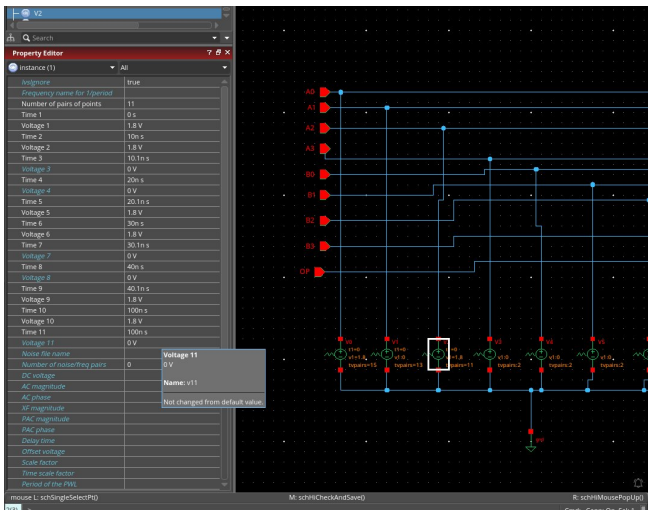
## 6.3 vpwl entry errors

**V5 (B1).** Voltage 5 was left at its default of 0 V instead of 1.8 V, so B1 would have dropped out of its high interval. The corrected entry (1.8 V) is shown in the V5 screenshot in Section 5.2.

**V2 (A2).** Time 11 was typed as 100n, the same as Time 10. Spectre stops with a fatal error because PWL times must strictly increase. Changing it to 100.1n fixes it.



*V5 before the fix: Voltage 5 = 0 V.*



*V2 before the fix: Time 11 = 100n, same as Time 10.*

```
ERROR (CMI-2204): `input.scs':156, V2: In the PWL waveform, time 100 ns, calculated using
expression `1e-07' at index 20, is less than the previously specified time 100 ns. ...
spectre terminated prematurely due to fatal error.
```

*Spectre log excerpt for the V2 error. It disappeared after the entry was corrected, and the full run completed.*

# 7. Notes and open items

- **Technology and supply.** The Spectre log shows the gpdk045 model files, simulated with a 1.8 V supply. The proposal text says gpdk180, L = 180 nm and VDD = 1.8 V. The report and proposal should name the technology that was actually used, and the W/L values should be taken from the schematics. gpdk045 devices are normally run at about 1.0 V, so the report should state why 1.8 V was used, or the supply can be lowered and the run repeated.
- **Glitches.** When several inputs change at the same instant, short spikes appear on the outputs (static hazards). They settle well before the end of each 10 ns window, and the results are read there.
- **Still to measure.** Worst-case carry delay (50% to 50%, B0 to R3 and to OVF, A = 1111, OP = 0, B0 pulsed), average power (VDD times average supply current over the 13-case run), and the transistor count taken from the final schematics.
- **Transistor counts** in Section 2 follow the proposal's decomposition. Confirm them against the real XOR2 and full-adder schematics.

## Conclusion

The slice passes all 16 input combinations and the 4-bit ALU passes all 13 planned cases in ADD and SUB, including the carry-out and borrow boundary cases. The design is verified at functional level; delay and power are the remaining measurements.